

Exam 27-01-2026 - Selected Solutions

Problem 1: Hospitals Distribution (Max-Flow)

(a) Polynomial-time Algorithm Il problema dell'assegnazione dei feriti agli ospedali può essere modellato e risolto in modo ottimo costruendo una rete di flusso (Flow Network). Un approccio *greedy* fallirebbe poiché prenderebbe decisioni locali senza considerare i colli di bottiglia globali.

Costruiamo un grafo orientato $G = (V, E)$ nel seguente modo: * **Nodi**: Aggiungiamo un nodo sorgente S , un nodo pozzo T , un nodo per ciascuna delle n persone $(p_1, \dots, p_n)$ e un nodo per ciascuno dei k ospedali $(h_1, \dots, h_k)$. * **Archi $S \rightarrow p_i$** : Creiamo un arco dalla sorgente S a ogni nodo persona p_i con capacità $c(S, p_i) = 1$. Questo garantisce che ogni ferito venga assegnato esattamente una volta. * **Archi $p_i \rightarrow h_j$** : Per ogni persona p_i e ogni ospedale h_j , controlliamo il vincolo temporale. Se la distanza è entro i 30 minuti ($dist(p_i, h_j) \leq 30$), aggiungiamo un arco di collegamento con capacità $c(p_i, h_j) = 1$. * **Archi $h_j \rightarrow T$** : Creiamo un arco da ogni ospedale h_j al pozzo T con capacità limitata a $c(h_j, T) = \lceil n/k \rceil$. Questo arco modella direttamente il vincolo di bilanciamento del carico.

Condizione di successo: Eseguiamo un algoritmo di Max-Flow da S a T . Se il flusso massimo ottenuto è esattamente uguale a n (ovvero tutte le n unità di flusso/persone riescono a raggiungere il pozzo senza violare i vincoli di capacità), l'algoritmo restituisce `TRUE`. Altrimenti, l'assegnazione non è fisicamente possibile e restituisce `FALSE`.

(b) Time Complexity Definiamo la dimensione del grafo in funzione dell'input: * Numero di vertici $|V| = n + k + 2 = O(n + k)$. * Numero di archi $|E| \leq n$ (da S) + $n \cdot k$ (tra persone e ospedali) + k (verso T) = $O(n \cdot k)$. * Il valore del flusso massimo da calcolare è limitato dal numero di persone, ovvero $f^* \leq n$.

Se utilizziamo l'algoritmo standard di Ford-Fulkerson, la complessità teorica è $O(|E| \cdot f^*)$. Sostituendo le nostre grandezze, otteniamo un tempo di $O(n \cdot k \cdot n) = O(n^2 k)$. Questo algoritmo opera quindi in tempo strettamente polinomiale rispetto alla dimensione dell'input.

Problem 2: Rod Cutting

(a) Dynamic Programming Algorithm Let $dp[i]$ be the maximum value obtainable by cutting a rod of length i . The base case is $dp[0] = 0$, since a rod of length 0 has value 0. For a rod of length $i > 0$, we can choose to make the first cut of length j (where $1 \leq j \leq i$), gaining value $v(j)$, and then optimally cutting the remaining rod of length $i - j$, which gives value $dp[i - j]$. To maximize the total value, we take the maximum over all possible choices of j :

$$dp[i] = \max_{1 \leq j \leq i} v(j) + dp[i - j]$$

(b) Pseudo-code and Time Complexity

```

Algorithm RodCutting(L, v):
    dp = array of size L+1 initialized to 0
    dp[0] = 0

    For i from 1 to L:
        max_val = 0
        For j from 1 to i:
            if v(j) + dp[i-j] > max_val:
                max_val = v(j) + dp[i-j]
        dp[i] = max_val

    Return dp[L]

## Problem 2: Rod Cutting

**(a) Dynamic Programming Algorithm**
Let $dp[i]$ be the maximum value obtainable by cutting a rod of length $i$.
The base case is $dp[0] = 0$, since a rod of length 0 has value 0.
For a rod of length $i > 0$, we can choose to make the first cut of length $j$ (where $1 \le j \le i$), gaining value $v(j)$, and then optimally cutting the remaining rod of length $i-j$.
$$$dp[i] = \max_{1 \le j \le i} \{ v(j) + dp[i-j] \}$$$

**(b) Pseudo-code and Time Complexity**
```text
Algorithm RodCutting(L, v):
 dp = array of size L+1 initialized to 0
 dp[0] = 0

 For i from 1 to L:
 max_val = 0
 For j from 1 to i:
 if v(j) + dp[i-j] > max_val:
 max_val = v(j) + dp[i-j]
 dp[i] = max_val

 Return dp[L]

```

**Time Complexity:** The algorithm uses due cicli annidati (nested loops). The outer loop runs  $L$  times. For each iteration of the outer loop, the inner loop runs  $i$  times. The total number of iterations is  $\sum_{i=1}^L i = \frac{L(L+1)}{2}$ . Inside the loop, all operations (addition, comparison) are executed in  $O(1)$  time. Pertanto, the overall time complexity is  $O(L^2)$ .

### Problem 3: Efficient Recruiting (NP-Completeness)

**1. Appartenenza a NP (Efficient Recruiting  $\in$  NP):** Il certificato (witness) è un sottoinsieme  $S$  di candidati scelti. Il verificatore deve controllare due cose in tempo polinomiale: 1. Che la dimensione del sottoinsieme sia  $|S| \leq k$ . 2. Che l'unione delle liste degli sport per cui i candidati in  $S$  sono qualificati contenga tutti gli  $n$  sport richiesti. Entrambe le operazioni richiedono tempo polinomiale limitato.

**2. NP-Hardness (Riduzione da Vertex Cover):** Costruiamo la riduzione a partire da un'istanza generica del Vertex Cover, ovvero un grafo non orientato  $G = (V, E)$  e un intero  $k$ . Costruiamo l'istanza dell'Efficient Recruiting come segue: \* **Candidati (c):** Creiamo un candidato per ogni singolo vertice del grafo  $G$  (dunque  $c = |V|$ ). \* **Sport (n):** Creiamo uno sport da coprire per ogni singolo arco del grafo  $G$  (dunque  $n = |E|$ ). \* **Competenze (Skill):** Un candidato (associato al vertice  $u$ ) è competente per uno sport (associato all'arco  $e$ ) se e solo se il vertice  $u$  è uno dei due estremi dell'arco  $e$  in  $G$ . \* **Target (k):** Il numero massimo di candidati da assumere rimane  $k$ . La costruzione richiede di scorrere archi e vertici del grafo originario e avviene chiaramente in tempo polinomiale.

**3. Dimostrazione (Se e Solo Se):** \* (  $\implies$  ) Se il grafo  $G$  ammette un Vertex Cover di dimensione  $\leq k$ , allora esiste un insieme di vertici  $V'$  tale per cui ogni arco in  $E$  è incidente ad almeno un vertice in  $V'$ . Selezionando i candidati corrispondenti a  $V'$ , avremo assunto al massimo  $k$  persone. Per costruzione delle competenze, poiché ogni arco è coperto, ogni sport avrà almeno un candidato qualificato. Dunque esiste una soluzione per l'Efficient Recruiting. \* (  $\impliedby$  ) Se abbiamo trovato una soluzione per l'Efficient Recruiting, ovvero un sottoinsieme  $S$  di al massimo  $k$  candidati in grado di coprire tutti gli sport, allora, seguendo la mappatura a ritroso, abbiamo selezionato al massimo  $k$  nodi. Il fatto che un candidato  $u$  sia competente per uno sport  $e$  significa che nel grafo esiste un arco  $e$  incidente in  $u$ . L'aver selezionato almeno un candidato qualificato per ogni sport implica aver selezionato almeno un estremo per ogni arco in  $E$ . Quindi,  $S$  corrisponde a un Vertex Cover valido per  $G$ .

### Problem 4: Directed Max-Cut Problem (Randomized Approximation)

**1. Definizione dell'algoritmo** Possiamo definire un algoritmo che scorre tutti i nodi del grafo  $G$  e assegna a random con probabilità  $p = \frac{1}{2}$  il nodo  $i$  a  $S$  oppure a  $T = V/S$ . Ovviamente la soluzione ottima, ovvero il massimo peso ottenibile selezionando gli archi per cui un estremo è in una partizione e l'altra estremo è nell'altra, non può essere maggiore della somma di tutti i pesi degli archi,  $OPT \leq \sum_{e \in E} w_e$ . Se la probabilità che un nodo finisca in una partizione è  $1/2$ , allora la probabilità che un arco venga selezionato è uguale alla probabilità che i nodi finiscano in diverse partizioni, i.e.  $\frac{1}{4}$ .

Considerando tutti gli archi, per linearità del valore atteso:  $\mathbb{E}[ALG] = \sum_{e \in E} w_e \Pr[e \text{ viene scelto}] = \frac{1}{4} \sum_{e \in E} w_e \geq \frac{1}{4} OPT$  Da cui otteniamo il rapporto di approssimazione.